

Deep Learning for NLP

Feedforward Neural Networks for NLP

From Feature Vectors → Neural Networks

NEURONS • LAYERS • WEIGHTS • BIAS • FORWARD PROPAGATION

Where Are We Starting?

From Module I's classical NLP pipeline to neural learning.

MODULE I

Text → Preprocessing → BoW / TF-IDF → Feature Vector → Classifier → Prediction

CLASSICAL

Naïve Bayes
Perceptron
Logistic Regression

TODAY

Replace the simple classifier with a neural network.

Feature Vector → Learned Intermediate Representation → Prediction

Learning Objectives

What students should be able to explain and calculate after this lecture.

BY THE END OF THE LECTURE

- Explain why neural networks are useful for NLP.
- Describe an artificial neuron.
- Explain weights and bias.
- Calculate a neuron's weighted sum.
- Understand layers and forward propagation.
- Distinguish parameters from activations.
- Formulate binary classification.
- Understand a feedforward NLP classifier.

Why Deep Learning for NLP?

Why move beyond a fixed linear classifier?

CLASSICAL

Text → TF-IDF → Logistic Regression

Feature representation is largely fixed before classification.

NEURAL

Input representation → learned intermediate features → prediction

The model can learn useful feature transformations rather than relying entirely on a fixed linear decision function.

From Logistic Regression to a Neural Network

A neural network adds learned intermediate transformations.

LOGISTIC REGRESSION

$$z = w^T x + b$$
$$\hat{y} = \sigma(z)$$

Direct mapping from input to output.

FEEDFORWARD NETWORK

$x \rightarrow$ hidden representation $\rightarrow$ output

One or more intermediate layers.

The neural network adds learned transformations between x and the final prediction.

What Is an Artificial Neuron?

The basic computational unit of a feedforward network.

INPUTS

x_1 x_2 x_3

Numerical features or activations from the previous layer.

NEURON

1. Multiply by weights
2. Add
3. Add bias
4. Apply activation

Inputs → weighted sum → activation → output

Mathematical Model of a Neuron

Weighted sum + bias + activation.

$$z = \sum_i w_i x_i + b$$

$$a = f(z)$$

x_i

input

w_i

weight

b

bias

f

activation

Understanding the Weight

How strongly an input influences the neuron.

POSITIVE

+2.1

Strengthens the response when the input increases.

NEGATIVE

-2.4

Pushes the weighted sum in the opposite direction.

NEAR ZERO

≈ 0

Relatively little influence.

Weights determine how strongly each input influences the neuron.

Understanding the Bias

An adjustable baseline or offset.

$$z = w^T x + b$$

WITHOUT BIAS

$$z = w^T x$$

Transformation constrained to pass through the origin.

WITH BIAS

$$z = w^T x + b$$

Bias provides an adjustable offset.

Think of bias as an adjustable baseline or offset.

Worked Neuron Example

A complete calculation inside one neuron.

INPUTS

$$\begin{aligned}x_1 &= 2 \\x_2 &= 3\end{aligned}$$

PARAMETERS

$$\begin{aligned}w_1 &= 0.5 \\w_2 &= -0.2 \\b &= 0.1\end{aligned}$$

CALCULATION

$$\begin{aligned}z &= (0.5)(2) + (-0.2)(3) + 0.1 \\z &= 0.5\end{aligned}$$

The activation function is applied after the weighted sum.

What Is a Layer?

A layer is a collection of neurons operating at the same stage.

INPUT LAYER

Receives the input vector.

HIDDEN LAYER

Transforms information through neurons.

OUTPUT LAYER

Produces the final prediction.

Input → Hidden → Output

Input Layer

The input layer receives the numerical representation of the text.

EXAMPLE TF-IDF VECTOR

[0.00, 0.42, 0.17, 0.00, 0.73, ...]

ROLE

Each dimension corresponds to a feature.

The input layer receives numbers; it does not itself “understand” language.

Hidden Layer

The hidden layer transforms the input.

COMPUTATION

$$z = Wx + b$$

$$a = f(z)$$

ROLE

Different hidden neurons can learn different combinations of input features.

Hidden layers create learned intermediate representations.

Output Layer

The output layer produces the final prediction.

BINARY

One output score / probability

Example: Positive = 0.91

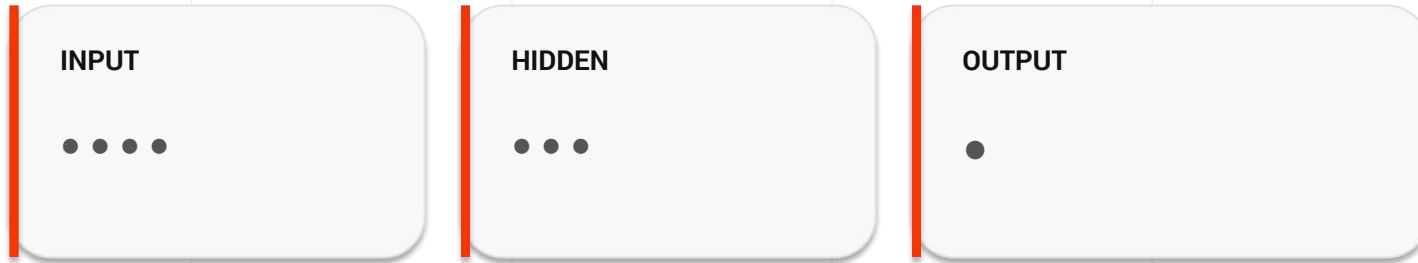
MULTICLASS

Positive 0.87
Negative 0.09
Neutral 0.04

Output-layer design depends on the NLP task.

A Complete Feedforward Network

Information moves from input to output.



INPUT → HIDDEN LAYER → OUTPUT

Information flows from input toward output. A basic feedforward network has no feedback cycles.

What Does “Feedforward” Mean?

Information moves in one direction.

DIRECTION

Input → Layer 1 → Layer 2 → Output

FEEDFORWARD

No cycles. Information moves in one direction.

CONTRAST

Recurrent architectures can carry information across sequence steps.

Forward Propagation

Computing the network's output from an input.

STEP 01

Input

STEP 02

Weighted Sum

STEP 03

Activation

STEP 04

Next Layer

STEP 05

Weighted Sum

STEP 06

Output

Question: "What does the current network predict?"

Two-Layer Mathematical Form

A single hidden layer written mathematically.

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{a}^{(1)} = f(\mathbf{z}^{(1)})$$

$$\mathbf{z}^{(2)} = \mathbf{W}^{(2)} \mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

$$\hat{\mathbf{y}} = g(\mathbf{z}^{(2)})$$

One hidden layer: transform → activate → transform → output.

Matrix Form

Why matrix operations matter in neural-network computation.

$$\mathbf{z} = \mathbf{W} \mathbf{x} + \mathbf{b}$$

VECTOR $\mathbf{x}$

Input features represented together.

MATRIX $\mathbf{W}$

Weights for connections between layers.

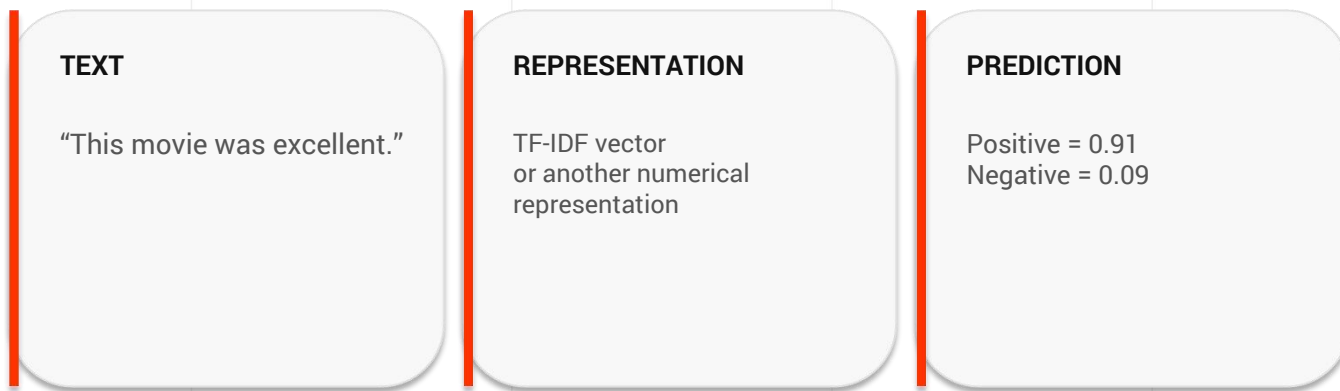
VECTOR $\mathbf{b}$

One bias per neuron.

Matrix operations make neural-network computation efficient.

Neural Network for Sentiment Analysis

A running NLP example: sentiment classification.



Text → Representation → Feedforward Network → Sentiment

Example Feature Interpretation

Learned combinations of input features.

FEATURES

excellent
amazing
boring
terrible

HIDDEN SIGNAL

A neuron may respond to a combination such as:

excellent + amazing

CAUTION

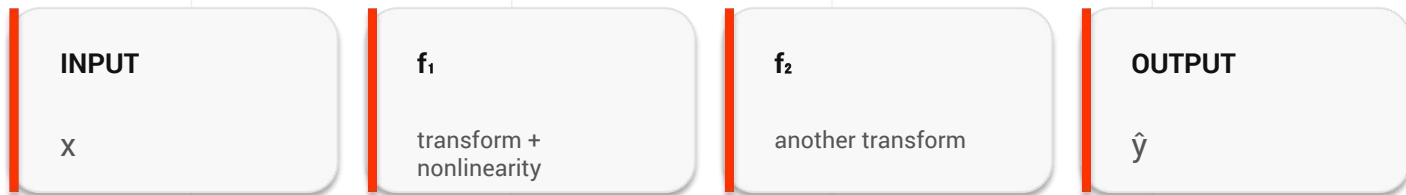
Hidden neurons do not necessarily correspond to clean, human-defined concepts.

The network learns combinations of features before making the final decision.

Neural Network as a Function

A feedforward network is a composition of functions.

$$f(x) = f_3 (f_2 (f_1 (x)))$$



A deep network is a composition of functions.

Why Multiple Layers?

Multiple nonlinear layers allow more complex transformations.

ONE LINEAR LAYER

Can learn a linear decision boundary.

Limited for complex nonlinear relationships.

MULTIPLE NONLINEAR LAYERS

Can learn increasingly complex transformations and feature combinations.

This is the fundamental motivation behind deep learning.

Number of Parameters

Calculating trainable parameters.

INPUT → HIDDEN

$5 \times 4 = 20$ weights
4 biases

24 parameters

HIDDEN → OUTPUT

$4 \times 1 = 4$ weights
1 bias

5 parameters

TOTAL

$24 + 5$

= 29 trainable
parameters

Parameters vs Hyperparameters

What the network learns versus what the practitioner chooses.

PARAMETERS

Learned during training:

- Weights
- Biases

HYPERPARAMETERS

Chosen by practitioner:

- Layers • Neurons • Learning rate • Batch size • Dropout • Epochs

Parameters are learned; hyperparameters are configured.

What Happens During Training?

Forward pass today; learning comes next.

01
Input

02
Forward propagation

03
Prediction

04
Loss

05
Gradient calculation

06
Parameter update

07
Repeat

Today: forward propagation • Later: how the network learns its parameters.

Binary Classification Output

Binary sentiment classification.

ARCHITECTURE

Input → Hidden Layers → Output Neuron → Sigmoid

EXAMPLE

$P(\text{Positive}) = 0.91$

Threshold = 0.5

$0.91 \geq 0.5 \rightarrow \text{Positive}$

For binary sentiment classification, the output is commonly interpreted as a probability.

Multiclass Classification

Multiclass classification.

TASK

Sports
Business
Politics
Technology

OUTPUT SCORES

Sports 0.10
Business 0.15
Politics 0.05
Technology 0.70

Predicted class → Technology

Softmax is commonly used for mutually exclusive multiclass outputs.

Simple Python Example

A compact one-neuron Python demonstration.

PYTHON – ONE NEURON

```
import numpy as np

x = np.array([2, 3])
w = np.array([0.5, -0.2])
b = 0.1

z = np.dot(w, x) + b
print("Weighted sum:", z)
```

Output: Weighted sum: 0.5

Summary & Next Lecture

The conceptual bridge to activation functions.

ARTIFICIAL NEURON

$$z = w^T x + b$$

$$a = f(z)$$

Weights + bias + activation.

FEEDFORWARD NETWORK

Input $\rightarrow$ Hidden Layer(s) $\rightarrow$ Output

Forward propagation computes the prediction.

NEXT LECTURE

Activation Functions: Sigmoid • Tanh • ReLU • Softmax